

Python Functions Reference for LeetCode Prep

Every Python function, method, and module used across the prep materials, explained with signatures, behavior, complexity, and worked examples. Organized by category.

1. Built-in Functions

These are always available — no import needed.

`len(obj)`

Returns the number of items in a container (list, string, dict, set, tuple).

- **Complexity:** $O(1)$ for built-in types.
- **Example:** `len([1, 2, 3])` → 3.

`range(start, stop, step)`

Produces an iterator over integers from `start` (inclusive) to `stop` (exclusive), stepping by `step`.

- `range(n)` → 0, 1, ..., $n-1$.
- `range(1, 5)` → 1, 2, 3, 4.
- `range(10, 0, -1)` → 10, 9, 8, ..., 1.
- **Complexity:** $O(1)$ creation; iterating consumes it lazily.
- **LeetCode use:** the standard way to iterate indices.

`enumerate(iterable, start=0)`

Yields `(index, value)` pairs from an iterable.

- **Example:**

```
for i, x in enumerate(['a', 'b', 'c']):
```

```
print(i, x) # 0 a, 1 b, 2 c
```

- **LeetCode use:** when you need both the index and the value while iterating. Cleaner than `for i in range(len(arr))`.

```
zip(*iterables)
```

Pairs up elements from multiple iterables.

- **Example:** `list(zip([1,2,3], ['a','b','c']))` → `[(1,'a'), (2,'b'), (3,'c')]`.
- **Stops at the shortest iterable.** Use `itertools.zip_longest` if you need full length.
- **Transpose trick:** `list(zip(*grid))` transposes a 2D list.

```
max(iterable) / max(a, b, c, ...)
```

Returns the largest value.

- With a `key` argument: `max(words, key=len)` returns the longest word.
- **LeetCode use:** constantly. Know `max(iterable, default=0)` to avoid errors on empty inputs.

```
min(iterable) / min(a, b, c, ...)
```

Returns the smallest value. Mirrors `max`.

```
sum(iterable, start=0)
```

Adds up an iterable of numbers.

- `sum([1, 2, 3])` → `6`.
- `sum(arr, 0.0)` if you want floats.
- **Not for strings** — use `"".join()` instead.

```
sorted(iterable, key=None, reverse=False)
```

Returns a *new* sorted list (doesn't modify the input).

- **Complexity:** $O(n \log n)$, Timsort.

- **Key function:** `sorted(words, key=lambda w: (len(w), w))` sorts by length then alphabetically.
- **Reverse:** `sorted(arr, reverse=True)` for descending.
- **Stable:** equal elements retain their original order.

Contrast with `list.sort()`, which sorts in place.

`reversed(iterable)`

Returns a reverse iterator. Use `list(reversed(arr))` if you need a list.

- **For lists:** faster than `arr[::-1]` when you only need to iterate (no extra memory).

`abs(x)`

Absolute value. Works for ints, floats, and complex numbers.

`pow(base, exp, mod=None)`

Exponentiation. The three-argument form is modular exponentiation, computed efficiently.

- `pow(2, 10) → 1024`.
- `pow(2, 100, 1000000007) → computes  $2^{100} \bmod (10^9+7)$  in  $O(\log \text{exp})$  time.`
- **Critical for competitive programming** — much faster than `(a**b) % p`.

`divmod(a, b)`

Returns `(a // b, a % b)` as a tuple. Useful when you need both quotient and remainder.

`round(x, ndigits=0)`

Rounds to a specified number of decimal places. Uses banker's rounding (round half to even).

`map(func, iterable)`

Applies a function to each element. Returns an iterator.

- `list(map(int, ['1', '2', '3'])) → [1, 2, 3]`.

- Often replaced by list comprehensions in Python 3 for readability: `[int(x) for x in strs]`.

`filter(func, iterable)`

Keeps elements where `func(x)` is truthy.

- `list(filter(lambda x: x > 0, [-1, 2, -3, 4])) → [2, 4]`.
- Often replaced by: `[x for x in arr if x > 0]`.

`any(iterable)` / `all(iterable)`

- `any` returns True if at least one element is truthy.
- `all` returns True if all elements are truthy (vacuously True on empty).
- **Short-circuit:** both stop early when the answer is determined.
- **Example:** `all(x > 0 for x in arr)` — is every element positive?

`list(iterable)` / `tuple(iterable)` / `set(iterable)` / `dict(iterable)`

Constructors that convert between types.

- `set(arr)` deduplicates.
- `list(range(5)) → [0, 1, 2, 3, 4]`.

2. List Methods

Methods on Python's built-in `list`. Most mutate the list in place.

`list.append(x)`

Adds `x` to the end. **O(1) amortized.**

`list.extend(iterable)`

Adds all elements of an iterable. Equivalent to `list += iterable`. **O(k)** where **k** is the length of the added iterable.

```
list.insert(i, x)
```

Inserts `x` at index `i`, shifting later elements right. **$O(n)$** . Avoid in tight loops.

```
list.pop(i=-1)
```

Removes and returns the element at index `i` (default last).

- `list.pop()` is $O(1)$.
- `list.pop(0)` is $O(n)$ — use `collections.deque` if you need to pop from the front.

```
list.remove(x)
```

Removes the first occurrence of `x`. $O(n)$.

```
list.index(x, start=0, stop=len)
```

Returns the index of the first occurrence of `x`. Raises `ValueError` if not found. $O(n)$.

```
list.count(x)
```

Counts occurrences of `x`. $O(n)$.

```
list.sort(key=None, reverse=False)
```

Sorts the list **in place**. $O(n \log n)$.

- Contrast with `sorted()`, which returns a new list.

```
list.reverse()
```

Reverses the list in place. $O(n)$.

```
list.copy() / arr[:] / list(arr)
```

Shallow copy. Doesn't copy nested objects.

```
list.clear()
```

Empties the list in place. Equivalent to `del arr[:]`.

3. String Methods

Strings are **immutable** — these all return new strings.

```
str.split(sep=None, maxsplit=-1)
```

Splits on whitespace by default, or on the given separator.

- `"a,b,c".split(",")` → `['a', 'b', 'c']`.
- `"hello world".split()` → `['hello', 'world']`.

```
str.join(iterable)
```

Concatenates an iterable of strings, separated by `str`.

- `",".join(['a', 'b', 'c'])` → `'a,b,c'`.
- **The idiomatic way to build strings** — $O(n)$ total, vs. $O(n^2)$ with `+=`.

```
str.strip() / lstrip() / rstrip()
```

Removes leading/trailing whitespace (or specified characters).

```
str.lower() / str.upper()
```

Case conversion.

```
str.replace(old, new, count=-1)
```

Replaces occurrences.

- `"hello".replace("l", "L")` → `"heLLo"`.

```
str.find(sub) / str.index(sub)
```

Find returns `-1` if not found; index raises `ValueError`.

```
str.startswith(prefix) / str.endswith(suffix)
```

Boolean checks. Can take tuples: `s.startswith(("http://", "https://"))`.

`str.isdigit()` / `str.isalpha()` / `str.isalnum()`

Boolean character-class checks.

`str.count(sub)`

Counts non-overlapping occurrences of `sub`.

`ord(ch)` / `chr(i)`

- `ord('A') → 65`.
 - `chr(97) → 'a'`.
 - Useful for character arithmetic: `ord(ch) - ord('a')` gives 0-indexed position.
-

4. Dictionary Methods

`dict.get(key, default=None)`

Returns the value for `key`, or `default` if absent. Doesn't raise `KeyError`.

- **LeetCode pattern:** `freq[x] = freq.get(x, 0) + 1` (or use `defaultdict`).

`dict.setdefault(key, default)`

Returns the value if present; otherwise inserts `default` and returns it.

- `groups.setdefault(key, []).append(word)` — the classic pre-`defaultdict` idiom.

`dict.items()` / `dict.keys()` / `dict.values()`

Views over the dictionary. Iterable; update live if the dict changes.

`dict.pop(key, default=...)`

Removes and returns the value. With `default`, doesn't raise if absent.

```
dict.update(other_dict)
```

Merges another dict's entries in, overwriting on conflict.

```
dict.fromkeys(iterable, value=None)
```

Creates a dict with keys from the iterable, all mapped to `value`.

- `dict.fromkeys([1,2,3], 0) → {1:0, 2:0, 3:0}`.
 - **Preserves order** since Python 3.7, useful for dedupe: `list(dict.fromkeys(arr))`.
-

5. Set Methods

```
set.add(x) / set.remove(x) / set.discard(x)
```

- `add` inserts.
- `remove` raises `KeyError` if absent.
- `discard` silently does nothing if absent.

```
set.union(other) / |
```

Elements in either set.

```
set.intersection(other) / &
```

Elements in both.

```
set.difference(other) / -
```

Elements in first but not second.

```
set.symmetric_difference(other) / ^
```

Elements in exactly one.

```
set.issubset(other) / <=
```


True if every element of self is in other.

`frozenset(iterable)`

Immutable version of a set. Can be used as a dict key or set element.

6. `collections` Module

Hugely important for LeetCode. Import with `from collections import ...`

`Counter(iterable)`

A dict subclass for counting hashable items.

```
from collections import Counter
c = Counter("abracadabra") # Counter({'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1})
c.most_common(2) # [('a', 5), ('b', 2)]
```

Key operations:

- Addition/subtraction: `c1 + c2`, `c1 - c2` (keeps only positive counts).
- `&` (intersection, min counts), `|` (union, max counts).
- `c.most_common(k)` returns top-k in $O(n \log k)$.

Use for: frequency counting, anagram checks, sliding window character counts.

`defaultdict(default_factory)`

A dict where missing keys auto-initialize via `default_factory()`.

```
from collections import defaultdict
groups = defaultdict(list)
groups['a'].append(1) # no KeyError, creates groups['a'] = [] then appends
```

Common factories: `int` ($\rightarrow 0$), `list`, `set`, `dict`, `lambda: ...`.

LeetCode use: building adjacency lists, grouping, frequency counting.

```
deque(iterable=None, maxlen=None)
```

Double-ended queue with $O(1)$ append/pop at both ends.

```
from collections import deque
dq = deque([1, 2, 3])
dq.append(4)          # right end: [1,2,3,4]
dq.appendleft(0)      # left end:  [0,1,2,3,4]
dq.pop()              # right end: returns 4
dq.popleft()          # left end:  returns 0
```

maxlen parameter gives a bounded deque — pushing onto a full deque discards from the other end. Useful for keeping a “last N” window.

LeetCode use: BFS queue, sliding window maximum, any problem needing $O(1)$ front operations.

```
OrderedDict
```

Preserves insertion order. Since Python 3.7, regular dicts do too, so `OrderedDict` is mainly needed for:

- `move_to_end(key)` — useful for LRU cache.
- Equality comparison that considers order.

7. `heapq` Module

Binary min-heap operations on a regular Python list. Import with `import heapq`.

```
heapq.heappush(heap, item)
```

Adds an item, maintaining heap invariant. $O(\log n)$.

```
heapq.heappop(heap)
```

Removes and returns the smallest item. $O(\log n)$.

```
heapq.heapify(list)
```

Converts a list into a heap in place. **$O(n)$** (better than pushing n times for $O(n \log n)$).

```
heapq.heappushpop(heap, item)
```

Pushes then pops in a single operation. More efficient than separate calls.

```
heapq.heapreplace(heap, item)
```

Pops then pushes. Assumes heap is non-empty.

```
heapq.nlargest(k, iterable, key=None) / nsmallest(k, iterable, key=None)
```

Returns top- k (or bottom- k) elements. $O(n \log k)$.

- **Faster than** `sorted(arr)[-k:]` when $k \ll n$.

```
heapq.merge(*iterables)
```

Merges sorted iterables into a single sorted iterator. Useful for external sorting.

Max-heap trick: Python only has a min-heap. To simulate max-heap, push $-x$ and negate on pop. For tuples, negate the sort key: `heapq.heappush(heap, (-priority, item))`.

8. `itertools` Module

```
itertools.combinations(iterable, r)
```

All r -length combinations, no repetition, in sorted order.

- `list(combinations([1,2,3], 2))` $\rightarrow$ `[(1,2), (1,3), (2,3)]`.
- Count: $C(n, r)$.

```
itertools.permutations(iterable, r=None)
```

All r -length permutations (r defaults to full length).

- `list(permutations([1,2,3]))` $\rightarrow$ all 6 orderings.
- Count: $n!/(n-r)!$.

```
itertools.product(*iterables, repeat=1)
```

Cartesian product.

- `list(product([1,2], ['a','b']))` → `[(1,'a'), (1,'b'), (2,'a'), (2,'b')]` .
- `product(range(2), repeat=3)` gives all 8 binary strings of length 3.

```
itertools.accumulate(iterable, func=operator.add)
```

Cumulative sum (or other operation).

- `list(accumulate([1,2,3,4]))` → `[1, 3, 6, 10]` .
- With `max` : running maximum.
- **LeetCode use:** building prefix sums in one line.

```
itertools.chain(*iterables)
```

Chains iterables together. `list(chain([1,2], [3,4]))` → `[1,2,3,4]` .

```
itertools.groupby(iterable, key=None)
```

Groups consecutive equal elements. **Requires sorted input for global grouping.**

- `[(k, list(g)) for k, g in groupby([1,1,2,2,3])]` → `[(1,[1,1]), (2,[2,2]), (3,[3])]` .

```
itertools.count(start=0, step=1)
```

Infinite counter. Useful with `zip` .

9. `functools` Module

```
functools.lru_cache(maxsize=128)
```

Memoization decorator.

```
from functools import lru_cache
```

```
@lru_cache(maxsize=None)  # None = unlimited cache
def fib(n):
    if n < 2: return n
    return fib(n-1) + fib(n-2)
```

Turns exponential recursion into linear. Essential for top-down DP. `maxsize=None` removes the limit.

Since Python 3.9: `@functools.cache` is the same as `@lru_cache(maxsize=None)` but faster.

`functools.reduce(func, iterable, initial=...)`

Applies `func` cumulatively. `reduce(lambda x, y: x*y, [1,2,3,4]) → 24` (product).

`functools.cmp_to_key(compare_fn)`

Converts a cmp-style comparison function (returns negative/0/positive) into a key for `sort`.

```
from functools import cmp_to_key
# Sort so concatenation is largest
def cmp(a, b):
    if a + b > b + a: return -1  # a first
    else: return 1               # b first
arr.sort(key=cmp_to_key(cmp))
```

`functools.partial(func, *args)`

Fixes some arguments of a function, returning a new callable.

10. `bisect` Module

Binary search on sorted lists. Import with `import bisect`.

`bisect.bisect_left(arr, x)`

Returns the leftmost index where you could insert `x` to keep `arr` sorted. $O(\log n)$.

- If `x` exists, returns the first position of `x`.

`bisect.bisect_right(arr, x)` **(same as `bisect.bisect`)**

Returns the rightmost valid insertion position.

- If `x` exists, returns one past the last position of `x`.

Difference of the two tells you the count of occurrences: `bisect_right - bisect_left`.

`bisect.insort_left(arr, x)` / `insort_right(arr, x)`

Inserts in sorted order. The binary search is $O(\log n)$ but the insert itself is $O(n)$. Useful but not asymptotically better than just sorting.

LeetCode use: maintain a sorted list of values seen so far; query for predecessors/successors; count elements in a range.

11. `math` Module

`math.floor(x)` / `math.ceil(x)`

Round down / up. For ints, use `//` for floor division and `-(-a // b)` for ceiling division.

`math.sqrt(x)` / `math.isqrt(x)`

- `sqrt` returns float. Subject to precision issues.
- `isqrt` (3.8+) returns integer square root; exact.

`math.log(x, base=e)` / `math.log2(x)` / `math.log10(x)`

Natural/binary/common logarithms.

`math.gcd(a, b)` / `math.lcm(a, b)`

Greatest common divisor and least common multiple.

`math.factorial(n)`

`n!` — exact integer.

`math.comb(n, r)` / `math.perm(n, r)`

Combinations C(n, r) and permutations P(n, r). Python 3.8+.

`math.inf` / `math.nan` / `math.isinf(x)` / `math.isnan(x)`

Infinity and NaN constants and checks.

`math.exp(x)` / `math.pi` / `math.e`

Exponentials and constants.

12. `random` Module

`random.random()`

Float in [0.0, 1.0).

`random.randint(a, b)`

Integer in [a, b], inclusive on both ends.

`random.randrange(start, stop, step)`

Like `range` but picks one. Stop is exclusive.

`random.choice(seq)`

A random element.

`random.choices(seq, weights=None, k=1)`

Sample with replacement, optionally weighted.

`random.sample(seq, k)`

Sample without replacement.

```
random.shuffle(list)
```

Shuffles in place.

13. Slicing and Indexing

```
arr[i:j:k]
```

Slice: start *i*, stop *j* (exclusive), step *k*.

- `arr[2:5]` → indices 2, 3, 4.
- `arr[::-1]` → reversed copy.
- `arr[::2]` → every other element.
- Negative indices count from the end: `arr[-1]` is the last.

Important: slicing creates a copy ($O(\text{length of slice})$). Prefer indices when performance matters.

```
list[i] = x
```

Assignment — $O(1)$.

```
list[i:j] = iterable
```

Slice assignment — can change the list's length.

```
del list[i] / del list[i:j]
```

Removal. `del list[i]` is $O(n)$. `del list[i:j]` is also $O(n)$ in general.

14. String Formatting

f-strings (Python 3.6+)

```
name = "Erik"
```



```
age = 30
print(f"{name} is {age} years old")
print(f"{3.14159:.2f}") # "3.14"
print(f"{42:05d}") # "00042"
```

`str.format()` (older)

```
"{} is {}".format(name, age)
"{name} is {age}".format(name="Erik", age=30)
```

15. Common Idioms and Tricks Used

List comprehensions

```
squares = [x**2 for x in range(10)]
evens = [x for x in range(10) if x % 2 == 0]
grid = [[0] * n for _ in range(m)] # m×n 2D array
```

Gotcha: `[[0]*n]*m` creates `m` references to the *same* list, not `m` independent lists. Always use the comprehension form for 2D arrays.

Dict / set comprehensions

```
square_map = {x: x**2 for x in range(10)}
unique_squares = {x**2 for x in range(10)}
```

Generator expressions

```
total = sum(x**2 for x in range(10)) # no parens around gen expr when it's the o
```

Lazy evaluation — doesn't build a list in memory. Use in `sum`, `max`, `any`, `all`, etc.

Unpacking

```
a, b, c = [1, 2, 3]
```

```
first, *rest = [1, 2, 3, 4]    # first=1, rest=[2,3,4]
*init, last = [1, 2, 3, 4]    # init=[1,2,3], last=4
a, b = b, a                   # swap without temporary
```

Walrus operator := (Python 3.8+)

Assign inside expressions.

```
while (line := input()) != "quit":
    process(line)
```

Chained comparisons

```
if 0 <= i < n: # Pythonic: equivalent to (0 <= i) and (i < n)
```

Ternary expression

```
x = a if condition else b
```

Truthiness

- Falsy: `False`, `None`, `0`, `0.0`, `""`, `[]`, `{}`, `set()`.
- Everything else is truthy.

is VS ==

- `==` compares values.
- `is` compares object identity (same object in memory).
- Use `is None`, `not == None`.

16. Classes and OOP basics used

Defining a class

```
class Node:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

`__init__` is the constructor; `self` is the instance reference.

Magic methods worth knowing

- `__init__` : constructor.
- `__repr__` : unambiguous string representation (for debugging).
- `__eq__` : equality.
- `__hash__` : for use as dict key / set element.
- `__lt__` : less-than, for sorting and heaps.
- `__len__` : makes `len(obj)` work.
- `__iter__` / `__next__` : makes an object iterable.

17. Common Type Conversions

From	To	Method
int	str	<code>str(n)</code>
str	int	<code>int(s)</code>
str	float	<code>float(s)</code>
str	list of chars	<code>list(s)</code>
list of chars	str	<code>"".join(chars)</code>
int	binary string	<code>bin(n) → '0b1010'</code>
int	hex string	<code>hex(n)</code>
binary string	int	<code>int('1010', 2)</code>

int	char	chr(n)
char	int	ord(c)

18. Exception Handling

```
try:
    result = risky_operation()
except ValueError as e:
    print(f"Bad value: {e}")
except (TypeError, KeyError):
    # handle multiple exception types
    pass
else:
    # runs if no exception
    pass
finally:
    # always runs
    pass
```

LeetCode use: usually not needed, but useful when wrapping `int(s)` that might fail.

19. A Quick Cheat Reference: "I need to..."

Need	Tool
Count characters in a string	<code>Counter(s)</code>
Build an adjacency list	<code>defaultdict(list)</code>
Implement a queue	<code>collections.deque</code>
Get top-k largest	<code>heapq.nlargest(k, arr)</code>
Sort with custom comparator	<code>sorted(arr, key=cmp_to_key(cmp))</code>
Binary search in sorted array	<code>bisect.bisect_left</code>
Memoize a recursive function	<code>@functools.cache</code>

Iterate with index	<code>enumerate(arr)</code>
Pair up two lists	<code>zip(a, b)</code>
Transpose a 2D list	<code>list(zip(*matrix))</code>
Cumulative sum	<code>list(itertools.accumulate(arr))</code>
All pairs (i, j) with $i < j$	<code>itertools.combinations(range(n), 2)</code>
Ceiling division	<code>-(-a // b) or (a + b - 1) // b</code>
Modular exponentiation	<code>pow(a, b, mod)</code>
Deduplicate preserving order	<code>list(dict.fromkeys(arr))</code>
Flatten a 2D list	<code>[x for row in grid for x in row]</code>
Check if all/any satisfy	<code>all(...) / any(...)</code>
Reverse a string	<code>s[::-1]</code>
Split a string into words	<code>s.split()</code>
Character to 0-indexed position	<code>ord(ch) - ord('a')</code>

20. A Minimal “Always Import These” Template

For competitive Python, have these memorized as the opening of every file:

```
from collections import defaultdict, Counter, deque
from heapq import heappush, heappop, heapify, nlargest, nsmallest
from itertools import combinations, permutations, accumulate, product
from functools import lru_cache, cache, cmp_to_key
from bisect import bisect_left, bisect_right, insort
import math
import sys

sys.setrecursionlimit(10**6) # if you use deep recursion
```

On LeetCode you don’t need the `sys.setrecursionlimit` call most of the time — they raise it for you.

Closing tips

- **Python is slow but expressive.** You often trade a constant factor of speed for much faster coding. On a 90-minute test, that's usually the right trade.
- **Know which operations are $O(1)$ vs $O(n)$.** The biggest time-to-TLE mistake is using `list.pop(0)` in a loop or string concatenation with `+=`.
- **`@functools.cache` is your friend.** It converts many "how do I do DP?" problems into "write the recursion and memoize."
- **When in doubt, write the list comprehension.** It's almost always the right tool and reads like English.

This covers the Python you'll actually use on a quant-trading LeetCode assessment. Keep this file open in a side tab while you practice — the idioms become reflexive quickly.